Change Request Management Module (CRMM)

Change Request Management Module (CRMM)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Change Management Standard (SCMS)

Category: Governance & Enforcement

Subcategory: Software Change Management Governance

Type: Core Module

Governed Space: Software Change Request Governance

Module ID: SLGA-SCMS-CRMM-01

Origin ID: OOF-SLGA-SCMS-CRMM

Version: 1.0

Status: Canonical Open

Effective Date: 14.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: English

Canonical Definition

Change Request Management Module (CRMM) is the Core Module within Software Change Management Standard (SCMS) that governs how proposed, required, and emerging software changes are recognized, expressed, identified, recorded, classified, maintained, and directed toward appropriate assessment or disposition.

CRMM establishes a software change request as an identifiable governance

object representing a proposed or required alteration to an existing or intended software state.

It requires each change request to preserve its origin, purpose, affected

software objects, current-state reference, proposed modification, supporting evidence, urgency, known constraints, and initial governance status.

The module distinguishes the recognition of a need for change from proof

that the proposed solution is suitable, the acceptance of a request from authorization to implement it, and a technically generated modification from a legitimately established change request.

CRMM governs change requests originating from human actors, organizations,

operational incidents, security findings, dependency developments, automated systems, AI coding assistants, and autonomous agents.

It establishes the initial identity and continuity required for subsequent

impact assessment, coordination, lifecycle re-entry, implementation traceability, and closure.

Its principal governance output is the Software Change Request Profile.

Operational Role

CRMM establishes the entry point for software change governance.

It governs:

• Change request recognition

• Request identity

• Change origin

• Requesting actor identification

• Problem or opportunity description

• Change purpose

• Current-state reference

• Proposed modification

• Affected software object identification

• Preliminary change scope

• Supporting evidence

• Initial urgency and priority

• Known dependencies and constraints

• Request completeness

• Duplicate and related requests

• Request classification

• Request revision

• Request withdrawal

• Request deferral

• Initial disposition

• Handoff to change impact assessment

CRMM does not determine the full impact of a change, authorize implementation, coordinate all affected lifecycle activities, or verify the final change outcome.

Operational Question

How can a proposed or required software change become an identifiable, sufficiently defined, evidence-supported, and traceable governance request without confusing the recognition of a problem, the proposal of a solution, the urgency of a modification, or an automated recommendation with authorization to change software?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS Core Module Flow

Change Request Management Module (CRMM) → Change Impact Assessment Module

(CIAM) → Change Coordination Control Module (CHCM) → Lifecycle Re-entry Control Module (LRECM) → Change Closure Management Module (CCLM)

CRMM is the first Core Module within SCMS.

It establishes the change request identity and initial governance record.

Its outputs provide the basis for CIAM to assess consequences and for

subsequent modules to coordinate, route, verify, and close the change.

A request may be revised or reassessed during later stages without losing its original identity and historical continuity.

• Governed Space

Software Change Request Governance

This governed space concerns the recognition, formation, identification,

classification, maintenance, and initial disposition of software change requests.

It begins when a potential change requirement, problem, opportunity, or modification proposal becomes identifiable.

It includes requests originating from development, testing, security,

production operations, dependencies, external requirements, automated analysis, and autonomous agents.

It continues until the request has an appropriate initial disposition and

sufficient information for its next governance step, or has been legitimately rejected, deferred, withdrawn, or otherwise classified.

The request remains traceable throughout the broader software change

lifecycle.

Governed Object

The primary Governed Object is:

A Software Change Request representing an identifiable proposal or

requirement to alter a software object or software condition, associated with its origin, requesting actor, purpose, current-state reference, proposed change, preliminary scope, supporting evidence, and governance status.

The governed object must preserve, where applicable:

Change Request ID

Software Product ID

Request Origin

Requesting Actor Reference

Request Creation Time

Change Purpose

Problem or Opportunity Reference

Affected Software Objects

Current State Reference

Proposed Modification

Preliminary Scope

Supporting Evidence

Urgency Reference

Known Constraints

Related Request References

Request Status

Request Version

Scope

CRMM governs:

Change Request Formation

Change Request Identification

Request Origin Attribution

Requesting Actor Identification

Purpose Definition

Problem and Opportunity Recognition

Proposed Change Description

Affected Object Identification

Current-State Reference

Preliminary Scope Definition

Evidence Association

Urgency Recognition

Request Completeness

Duplicate Request Detection

Related Request Association

Request Classification

Request Revision

Request Disposition

Historical Request Continuity

Assessment Handoff

CRMM does not prescribe a particular ticketing system, issue tracker, repository platform, development methodology, or organizational approval workflow.

Foundational Principle

A software change request establishes what change is being proposed or required and why it deserves governance consideration; it does not establish that the change is suitable, authorized, or ready for implementation.

A defect report may justify investigation without proving that a particular correction is appropriate.

A security alert may establish urgency without authorizing unrestricted modification.

An automated system may identify a dependency problem without establishing the legitimacy of its proposed replacement.

An autonomous agent may generate a technically plausible patch without authority to introduce it into the software lifecycle.

Change Need Identified ≠ Proposed Solution Validated.

Change Request Accepted ≠ Change Implementation Authorized.

Change Request Identity

Every governed change request must have a stable identity.

Relevant elements include:

Change Request ID

Software Product ID

Origin Reference

Creation Time

Requesting Actor

Current Status

Request Version

Evidence Reference

A request identity must remain stable through material revisions, reassessment, deferral, or transfer.

Where a request is replaced or separated into multiple requests, the relationship must remain traceable.

Change Request Formation

A change request is formed when a recognizable change need or proposal is expressed in a manner suitable for governance assessment.

Minimum formation elements include:

Identifiable Software Context

Recognizable Change Origin

Stated Problem or Opportunity

Requested or Proposed Change

Known Affected Scope

Requesting Source

Initial Evidence or Justification

A request may enter governance with incomplete information.

Its incompleteness must remain explicit.

Change Origin

CRMM must identify the source from which the need for change emerged.

Potential origins include:

New Requirement

Defect Discovery

Security Vulnerability

Operational Incident

Dependency Change

Configuration Issue

Quality Finding

Performance Limitation

Compatibility Requirement

Infrastructure Change

External Obligation

Customer Feedback

Automated Detection

Autonomous Agent Recommendation

Software Retirement Requirement

The origin must remain distinguishable from the actor submitting the request.

Requesting Actor Identity

CRMM must identify the actor or system responsible for submitting or generating the request.

Potential actors include:

Developer

Software Maintainer

Quality Engineer

Security Function

Operations Function

Product Owner

Customer Representative

Automated Monitoring System

AI Coding Assistant

Autonomous Software Agent

External System

Where the source is automated, the responsible system and its assigned function must remain identifiable.

Origin–Actor Distinction

A software change request may originate from one condition while being submitted by another actor.

For example:

A monitoring system detects an operational anomaly.

An autonomous agent interprets the anomaly.

A human operator submits the formal change request.

CRMM must preserve the distinction between:

Change Origin

Detection Source

Requesting Actor

Proposed Solution Source

These identities must not be collapsed into a single undifferentiated requester field where doing so would obscure material provenance.

Problem Recognition

CRMM must establish the problem or condition motivating a corrective change request.

Relevant elements include:

Observed Condition

Expected Condition

Material Difference

Affected Software

Observation Time

Operational Context

Supporting Evidence

Known Uncertainty

A reported problem must remain distinguishable from a verified defect or established root cause.

Opportunity Recognition

Not every change request originates from a failure.

Potential improvement opportunities include:

New Functionality

Performance Improvement

Resource Efficiency

Usability Improvement

Compatibility Expansion

Technical Simplification

Maintainability Improvement

Dependency Modernization

Automation Improvement

Operational Capability Enhancement

An opportunity must still have an identifiable purpose and a sufficiently described intended outcome.

Change Purpose Definition

Every request must state what the proposed change is intended to accomplish.

Relevant elements include:

Purpose Statement

Problem or Opportunity

Expected Benefit

Affected Function

Intended Outcome

Success Reference

Known Constraints

A request that only specifies a technical action may require clarification of the underlying purpose.

Requested Action ≠ Established Change Purpose.

Requested Modification

CRMM must distinguish the requested modification from the underlying change need.

Examples include:

Modify Source Code

Replace Dependency

Update Configuration

Change Software Behavior

Introduce Functionality

Remove Functionality

Correct a Security Condition

Change an Interface

Update Firmware

Modify Deployment-Related Software Settings

The proposed modification may later be replaced by a different solution following impact assessment.

Current-State Reference

CRMM must preserve the available reference to the software state that motivates the request.

Relevant elements include:

Current Software Version

Current Artifact

Current Configuration

Current Dependency

Current Behavior

Current Operational Condition

Known Defect

Known Restriction

Evidence Time

The reference may initially be incomplete.

CRMM must identify missing information relevant to subsequent assessment.

Proposed-State Reference

Where possible, the request must describe the intended resulting state.

Relevant elements include:

Expected Functionality

Expected Behavior

Target Dependency Condition

Target Configuration

Target Compatibility

Target Security Condition

Target Performance

Expected Operational Outcome

The proposed state is not a verified implementation result.

Affected Software Object Identification

CRMM must identify the software objects believed to be affected.

Potential objects include:

Software Product

Application

Service

Source Component

Library

Dependency

Configuration

Build Artifact

Interface

Database Schema

Firmware

Automation Logic

Agent Behavior Configuration

Deployment-Related Software Object

Preliminary identification must not be treated as a complete impact assessment.

Preliminary Change Scope

CRMM establishes the scope proposed by the requester.

Relevant elements include:

Affected Software Objects

Requested Functional Changes

Known Dependency Relationships

Target Environments

Expected Data Changes

Known Exclusions

Implementation Constraints

Known Uncertainty

The scope remains provisional until appropriate impact assessment and

governance decisions establish its applicability.

Request Boundary

The request boundary identifies what the request includes and what remains

outside its stated purpose.

Relevant dimensions include:

Functional Boundary

Software Object Boundary

Dependency Boundary

Configuration Boundary

Data Boundary

Environment Boundary

Authority Boundary

Operational Boundary

A request must not become an unrestricted authorization to modify adjacent systems.

Supporting Evidence

CRMM must associate available evidence with the request.

Evidence may include:

Defect Report

Test Result

Security Finding

Monitoring Observation

Dependency Advisory

User Report

Compatibility Report

Performance Measurement

Operational Incident Record

Automated Analysis

Agent-Generated Assessment

Evidence must remain distinguishable from assumptions and proposed explanations.

Evidence Provenance

Material evidence must retain relevant provenance.

Relevant elements include:

Evidence ID

Evidence Source

Collection Method

Collection Time

Observed Software Object

Evidence Integrity Reference

Known Limitations

Verification Status

Automated or AI-generated evidence must not be treated as independently verified merely because it is presented in a structured report.

Evidence Sufficiency at Request Formation

CRMM evaluates whether the request contains sufficient information for its

next governance step.

Relevant questions include:

Is the software context identifiable?

Is the change purpose understandable?

Are affected objects sufficiently identified?

Is the proposed modification described?

Is supporting evidence available?

Are material uncertainties visible?

Can the request be meaningfully assessed?

A request may be valid for investigation without being ready for full impact assessment.

Request Completeness

CRMM recognizes the following completeness conditions:

Complete for Initial Assessment

Partially Complete

Clarification Required

Evidence Required

Scope Undefined

Origin Uncertain

Request Conflicted

Completeness is assessed relative to the intended next governance step.

It does not imply that the proposed change is feasible or authorized.

Change Request Classification

CRMM must establish an initial classification.

Potential categories include:

Functional Change Request

Corrective Change Request

Security Change Request

Dependency Change Request

Configuration Change Request

Compatibility Change Request

Performance Change Request

Emergency Change Request

Migration Change Request

Retirement-Related Change Request

Automated Change Request

Autonomous Agent-Initiated Change Request

A request may belong to multiple categories.

Classification may change following further assessment.

Request Urgency

CRMM must preserve the stated urgency of a request.

Relevant factors include:

Operational Severity

Security Exposure

Service Disruption

User Impact

External Deadline

Dependency Deadline

Safety-Relevant Condition

Continuity Requirement

Potential Consequence of Delay

Urgency must remain distinguishable from implementation authority.

Urgent Request ≠ Authorized Emergency Change.

Request Priority

Priority concerns the relative importance of processing a request.

Relevant elements include:

Requested Priority

Priority Justification

Known Impact

Urgency

Resource Considerations

Related Requests

Applicable Coordination Needs

The requester may propose priority.

The competent governance function determines applicable prioritization

according to its authority and criteria.

Request Justification

Request justification must identify why the proposed change deserves consideration.

Relevant elements include:

Problem Evidence

Expected Benefit

Operational Necessity

Applicable Requirement

Risk of Inaction

Known Alternatives

Time Sensitivity

Known Uncertainty

A persuasive justification does not independently establish solution suitability or implementation permission.

Request Assumptions

CRMM must preserve assumptions materially affecting the proposed change.

Examples include:

Assumed Root Cause

Assumed Dependency Compatibility

Assumed Environment State

Assumed Software Version

Assumed Resource Availability

Assumed Implementation Simplicity

Assumed Recovery Feasibility

Assumptions must remain distinguishable from verified facts.

Request Constraints

Known constraints may include:

Operational Availability Requirements

Security Restrictions

Compatibility Requirements

Resource Limits

Implementation Deadlines

Data Protection Conditions

Infrastructure Limitations

External Dependencies

Physical-System Restrictions

Constraints must be preserved for subsequent impact assessment and coordination.

Request Dependencies

A change request may depend on other changes or conditions.

Relevant elements include:

Prerequisite Change Request

Related Defect

Dependency Update

Configuration Revision

Required Infrastructure Change

External System Requirement

Required Migration

Required Recovery Action

Dependencies must remain identifiable even when their complete consequences are not yet known.

Duplicate Request Detection

Multiple requests may concern the same underlying change need.

CRMM must support identification of:

Identical Requests

Substantially Overlapping Requests

Related but Distinct Requests

Repeated Reports of One Defect

Different Solutions to One Problem

Duplicate detection must not erase independent evidence or distinct operational consequences.

Request Consolidation

Related requests may be consolidated where justified.

Consolidation must preserve:

Original Request IDs

Original Sources

Distinct Evidence

Affected Objects

Combined Purpose

Material Differences

Consolidation Decision

Historical References

Requests must not be merged merely because they share a broad software category.

Request Separation

A request may contain multiple materially distinct changes.

Separation may be required where changes differ in:

Purpose

Affected Software Objects

Impact

Authority

Urgency

Implementation Pathway

Verification Requirements

Operational Consequences

Separated requests must retain traceability to their common origin.

Request Relationship Governance

CRMM must preserve relationships among change requests.

Potential relationships include:

Duplicate Of

Related To

Depends On

Blocks

Supersedes

Superseded By

Derived From

Split From

Merged Into

These relationships support subsequent coordination without predetermining

implementation decisions.

Automated Request Generation

Automated systems may create change requests from predefined conditions.

Examples include:

Dependency Update Detection

Security Alert

Failed Quality Test

Configuration Drift

Operational Monitoring Trigger

Performance Threshold

Compatibility Failure

CRMM must preserve the generating system, detection basis, request scope, and evidence.

Automated Detection ≠ Validated Change Requirement.

AI-Generated Change Requests

AI systems may formulate change requests using code analysis, operational observations, documentation, or user instructions.

CRMM must identify:

AI System or Agent

Assigned Function

Input Context Reference

Proposed Change

Supporting Evidence

Generated Assumptions

Known Limitations

Required Review

AI-generated explanations must not be treated as established facts without appropriate verification.

Autonomous Agent Request Boundary

An autonomous agent may be authorized to identify or propose changes without being authorized to implement them.

CRMM must preserve:

Agent Identity

Assigned Objective

Request Generation Scope

Permitted Evidence Sources

Affected Software Objects

Decision Boundary

Escalation Requirements

Applicable Authority Reference

Agent Permission to Propose ≠ Agent Permission to Implement.

Human and Organizational Request Governance

Human-generated requests must also preserve applicable identity and authority distinctions.

A developer may identify a defect.

A product owner may request new functionality.

An operations engineer may report an unacceptable deployment condition.

A security function may require remediation.

These roles do not automatically possess identical authority to approve,

implement, release, or deploy the proposed changes.

Request Review

CRMM must establish an initial review sufficient to determine the request's next governance step.

Review may evaluate:

Request Identity

Purpose Clarity

Affected Object Identification

Evidence Availability

Preliminary Classification

Known Constraints

Related Requests

Material Uncertainty

Assessment Readiness

The review must not substitute for CIAM's substantive change impact assessment.

Request Clarification

Clarification may be required when the request lacks information necessary for meaningful assessment.

Examples include:

Unclear Problem Description

Undefined Software Product

Unknown Affected Component

Ambiguous Intended Outcome

Unsupported Urgency

Conflicting Evidence

Unclear Requesting Source

Missing Operational Context

Clarification requirements must remain identifiable and traceable.

Request Rejection

A request may be rejected at the request-management stage when an appropriate basis exists.

Potential reasons include:

Outside Governed Software Scope

Insufficient Justification After Reasonable Clarification

Request Withdrawn by Applicable Actor

Duplicate Already Governed Elsewhere

Proposed Change Incompatible With Mandatory Constraints

No Identifiable Change Need

Rejection must preserve its basis and applicable decision authority.

Rejection at this stage must not be used to conceal material defects, security findings, or operational risks that require governance through another mechanism.

Request Deferral

A request may be deferred when assessment or processing cannot appropriately proceed.

Potential reasons include:

Missing Evidence

Unresolved Dependency

Insufficient Context

Resource Constraint

External Decision Pending

Related Change Pending

Required Clarification

Deferral must preserve the reason, responsible function, and applicable reassessment condition.

Request Withdrawal

A request may be withdrawn when the applicable actor no longer seeks the proposed modification.

CRMM must preserve:

Withdrawal Actor

Withdrawal Authority or Eligibility

Withdrawal Reason

Withdrawal Time

Related Obligations

Remaining Risk References

Disposition Evidence

Withdrawal of a request must not automatically eliminate independently established safety, security, or operational obligations.

Request Supersession

A request may be superseded by a newer or more appropriate change request.

Relevant elements include:

Original Request

Successor Request

Reason for Supersession

Transferred Evidence

Changed Scope

Changed Purpose

Outstanding Obligations

Historical Continuity

Supersession must not erase the earlier request's decisions or evidence.

Request Revision

A request may require revision when its purpose, scope, evidence, or proposed modification changes.

CRMM must preserve:

Request Version

Previous Version

Revised Elements

Revision Reason

Revision Actor

Revision Time

New Evidence

Assessment Implications

Material revisions must not silently inherit earlier conclusions that no longer apply.

Request Status

CRMM recognizes the following principal request states:

Identified

Draft

Submitted

Clarification Required

Evidence Required

Under Initial Review

Complete for Initial Assessment

Ready for Impact Assessment

Deferred

Rejected

Withdrawn

Superseded

Transferred

Unknown

Conflicted

Request status must remain distinct from the broader change implementation and closure status governed by SCMS.

Initial Request Disposition

CRMM must establish an appropriate initial disposition.

Possible outcomes include:

Proceed to Impact Assessment

Request Clarification

Request Additional Evidence

Consolidate With Related Request

Separate Into Distinct Requests

Defer

Reject

Withdraw

Supersede

Transfer to Another Applicable Governance Mechanism

The disposition must remain associated with its basis and competent decision reference.

Request-to-Assessment Handoff

Where a request proceeds to CIAM, CRMM must provide sufficient information for substantive impact assessment.

The handoff should include:

Change Request ID

Software Product

Change Origin

Purpose

Current-State Reference

Proposed Modification

Affected Objects

Preliminary Scope

Evidence

Known Constraints

Known Dependencies

Uncertainty

Related Requests

Request Classification

The handoff must preserve the distinction between preliminary request information and verified assessment findings.

Request Evidence Continuity

CRMM must preserve material evidence throughout the request lifecycle.

This includes:

Original Submission

Supporting Evidence

Clarifications

Request Revisions

Duplicate Assessments

Relationship Decisions

Initial Reviews

Disposition Decisions

Handoff Records

Historical evidence must remain reconstructable even when the request is later revised, merged, or superseded.

Request Governance Classification

CRMM recognizes the following governance classifications:

Request Identifiable

Request Incomplete

Purpose Unclear

Scope Unclear

Evidence Insufficient

Origin Uncertain

Related Request Identified

Duplicate Candidate

Request Conflicted

Ready for Assessment

Deferred

Rejected

Withdrawn

Superseded

Transferred

Unknown

Classification must remain associated with the relevant request version and evidence.

Governance Requirements

A conforming CRMM implementation must satisfy the following requirements. Request Identity and Formation

• Establish a stable change request identity

• Identify the software product or relevant software context

• Preserve the request origin

• Identify the requesting actor or system

• Preserve the creation time

• Establish the change purpose

• Identify the proposed modification

• Preserve the request version

Scope and Context

• Identify the current-state reference

• Identify the intended change outcome where available

• Identify affected software objects

• Establish preliminary scope

• Preserve known exclusions

• Preserve material assumptions

• Preserve known constraints

• Identify related requests and dependencies

Evidence and Classification

• Preserve supporting evidence

• Identify evidence provenance

• Distinguish observations from assumptions

• Evaluate request completeness

• Identify missing evidence

• Establish initial change classification

• Preserve urgency justification

• Distinguish urgency from authority

Request Control

• Identify duplicate requests

• Preserve consolidation decisions

• Preserve request separation references

• Maintain request relationships

• Preserve material revisions

• Govern deferral and withdrawal

• Preserve rejection and supersession decisions

• Maintain historical request continuity

Automation and Handoff

• Identify automated request sources

• Preserve AI-generated request limitations

• Distinguish agent proposal authority from implementation authority

• Preserve initial review outcomes

• Establish request disposition

• Identify assessment readiness

• Provide structured handoff to CIAM

• Preserve the distinction between request acceptance and change authorization.

Governance Logic

Change Signal → Origin Identification → Requesting Actor → Software

Context → Problem or Opportunity → Change Purpose → Proposed Modification → Affected Objects → Preliminary Scope → Evidence → Assumptions and Constraints → Request Completeness → Initial Classification → Related Request Assessment → Initial Review → Request Disposition → CIAM Handoff / Clarification / Deferral / Rejection / Withdrawal / Supersession

Where information is insufficient:

Request → Missing Information → Clarification or Evidence Requirement →

Revised Request → Completeness Reassessment

Where duplicate or related requests exist:

Request → Relationship Assessment → Preserve Distinct Evidence →

Consolidate / Separate / Maintain Relationship

Where an autonomous agent generates a request:

Agent Observation → Proposed Change → Agent Identity and Scope → Evidence

Assessment → Request Formation → Authorized Review Pathway

Where the request changes materially:

Original Request → Material Revision → Version Preservation → Reassessment

of Initial Conclusions → Updated Disposition

CRMM must not convert request formation, automated detection, or preliminary acceptance into permission to implement software modifications.

Governance Outputs

The principal output is:

Software Change Request Profile

Supporting outputs may include:

Change Request Identity

Change Origin Record

Requesting Actor Reference

Change Purpose Statement

Problem or Opportunity Record

Current-State Reference

Proposed Modification Record

Affected Software Object Register

Preliminary Change Scope

Supporting Evidence Register

Request Completeness Assessment

Initial Change Classification

Urgency Reference

Known Constraint Register

Related Request Register

Request Revision History

Request Disposition Record

Assessment Readiness Classification

CIAM Handoff Record

Minimum Implementation Framework

A minimum CRMM implementation must establish eight governance layers. Layer 1 — Request Identity

Identify the change request, software context, requesting actor, origin,

and creation time. Layer 2 — Change Purpose

Establish the problem or opportunity, proposed modification, and intended

outcome. Layer 3 — Software Context

Identify affected software objects, current-state references, preliminary

scope, assumptions, and known constraints. Layer 4 — Evidence

Preserve supporting observations, reports, source references, provenance,

and material uncertainties. Layer 5 — Classification

Establish the initial request category, urgency, completeness, and known

related requests. Layer 6 — Request Control

Manage clarification, duplicate detection, consolidation, separation,

revision, deferral, withdrawal, rejection, and supersession. Layer 7 — Disposition

Determine whether the request is sufficiently defined for impact assessment or requires another legitimate disposition. Layer 8 — Governed Output and Handoff

Establish the Software Change Request Profile and preserve the necessary

information for CIAM.

The implementation may use manual forms, issue-tracking systems, repository workflows, automated detection, or AI-assisted request generation.

The governance logic remains independent of the technology used.

Cross-Module Boundary

CRMM → CIAM

CRMM establishes the request identity, purpose, preliminary scope, affected

objects, and available evidence.

CIAM evaluates the direct and indirect consequences, feasibility,

dependencies, and lifecycle implications of the proposed change.

CRMM → CHCM

CRMM preserves initial relationships among requests.

CHCM governs coordination among affected actors, activities, decisions, and

related changes.

CRMM → LRECM

CRMM identifies the proposed change.

LRECM determines the applicable lifecycle re-entry requirements based on

the assessed impact and governance conditions.

CRMM → CCLM

CRMM preserves the original request and its historical revisions.

CCLM evaluates the final change disposition, verified outcome, and closure

obligations.

Request Ready for Assessment ≠ Change Ready for Implementation.

Cross-Architecture Boundary

CRMM may receive change signals and evidence from:

SRS — Software Requirements Standard

New or revised software requirements.

SCS — Software Creation Standard

Development findings and artifact modification proposals.

SDS — Software Dependency Standard

Dependency changes, incompatibilities, and continuity requirements.

SCFS — Software Configuration Standard

Configuration changes, build findings, and artifact identity discrepancies.

SQS — Software Quality Standard

Defects, failed tests, and quality findings.

SSS — Software Security Standard

Vulnerabilities, security findings, and remediation requirements.

SRLS — Software Release Standard

Release readiness findings, release restrictions, and withdrawal-related

change needs.

SDPS — Software Deployment Standard

Deployment failures, production version inconsistencies, and

recovery-related change needs.

SRTS — Software Retirement Standard

Migration, replacement, and retirement-related modifications.

SLCS — Software Lifecycle Control Standard

Lifecycle inconsistencies, exceptions, and transition requirements.

CRMM establishes the change request associated with these signals.

It does not replace the governance mechanisms responsible for the originating conditions.

Practical Applications

Software Development Teams

Creating traceable requests for defects, enhancements, technical

improvements, and compatibility modifications.

Security Operations

Converting vulnerability findings into identifiable change requests without

confusing remediation urgency with implementation authorization.

Automated Dependency Management

Maintaining evidence-supported requests for dependency updates detected by

automated tools.

AI-Assisted Software Engineering

Preserving the identity, purpose, assumptions, and evidence of AI-generated

software modification proposals.

Autonomous Agent Systems

Separating an agent's ability to detect or propose changes from its

permission to modify software or production infrastructure.

Industrial and Embedded Software

Establishing traceable requests for firmware, controller configuration, and

software behavior changes.

Use Case 1 — An AI Agent Proposes a Security Fix Without Sufficient

Evidence

An autonomous coding agent analyzes a software repository and identifies a

dependency it considers vulnerable.

The agent proposes replacing the dependency with a newer version.

Its report contains a vulnerability claim but does not establish which dependency version is actually used in the relevant production release.

CRMM evaluates:

Agent Recommendation → Request Identity → Software Context → Vulnerability

Claim → Proposed Modification → Supporting Evidence → Request Completeness

The assessment identifies:

• The agent's proposal is a recognizable change request

• The software object and proposed modification are identifiable

• The production dependency reference is not sufficiently established

• The evidence does not yet support the full vulnerability claim

• Additional clarification or evidence is required before substantive

impact assessment.

The appropriate classification is:

Evidence Insufficient — Clarification Required.

AI-Generated Security Recommendation ≠ Verified Software Change

Requirement.

Use Case 2 — Multiple Teams Report the Same Defect With Different

Operational Evidence

Three teams submit requests concerning failures in a shared authentication

service.

The development team reports a failed test.

The operations team reports intermittent production authentication failures.

The customer support team reports user access problems.

All three requests appear to concern the same software component.

CRMM evaluates:

Request A → Request B → Request C → Shared Software Object → Evidence

Comparison → Relationship Assessment → Consolidation Decision

The assessment identifies:

• The requests may concern one underlying problem

• Each request contains distinct evidence

• The operational consequences differ across the reports

• Consolidation may be appropriate if the common change need is

• Original evidence and request identities must remain traceable

established.

The appropriate classification is:

Related Requests Identified — Consolidation Assessment Required.

Shared Software Component ≠ Automatically Identical Change Request.

Critical Distinctions

Change Need Identified ≠ Proposed Solution Validated.

Change Request Accepted ≠ Change Implementation Authorized.

Requested Action ≠ Established Change Purpose.

Reported Problem ≠ Verified Defect.

Observed Failure ≠ Established Root Cause.

Preliminary Scope ≠ Complete Change Impact.

Request Urgency ≠ Implementation Authority.

Urgent Request ≠ Authorized Emergency Change.

Supporting Evidence Available ≠ Evidence Sufficient.

AI-Generated Recommendation ≠ Verified Change Requirement.

Automated Detection ≠ Validated Change Requirement.

Agent Permission to Propose ≠ Agent Permission to Implement.

Related Requests ≠ Duplicate Requests.

Request Consolidation ≠ Evidence Elimination.

Request Withdrawal ≠ Underlying Risk Resolved.

Request Ready for Assessment ≠ Change Ready for Implementation.

Canonical Boundary

CRMM governs the recognition, identification, formation, preliminary

classification, evidence association, maintenance, and initial disposition of software change requests.

It establishes the relationship between a change origin, requesting actor,

software context, proposed modification, preliminary scope, supporting evidence, and request governance status.

CRMM does not determine complete change impact, authorize implementation, coordinate the full change lifecycle, or verify the resulting software state.

Its principal governance output is the:

Software Change Request Profile

This output establishes the identifiable and evidence-supported starting

point for subsequent software change governance.

Canonical Closing Statement

A software change cannot be governed effectively if the original reason

for the change, the proposed modification, and the evidence supporting the request cannot be distinguished.

A defect report may describe a real problem while proposing the wrong solution.

A security alert may justify urgent investigation without establishing the correct remediation.

A dependency update may appear routine while its operational consequences remain unknown.

Multiple teams may report the same underlying problem through different observations.

An AI agent may generate a convincing change proposal without possessing sufficient evidence or authority to implement it.

And a software modification may be technically possible before anyone has established why it should be made.

CRMM establishes the governance structure required to distinguish these

conditions.

It requires software change requests to remain identifiable, attributable,

sufficiently described, and connected to their supporting evidence.

It preserves uncertainty, preliminary assumptions, request relationships,

and material revisions.

It ensures that requests can be clarified, assessed, consolidated, deferred, rejected, or transferred without losing their origin and historical continuity.

And it establishes a disciplined boundary between recognizing a change need

and deciding whether or how the proposed change should proceed.

A change request is not permission to modify software. It is the identifiable governance starting point from which the necessity, consequences, legitimacy, and appropriate lifecycle pathway of a proposed modification can be established.

Change Request Management Module (CRMM) transforms software change

proposals from informal instructions or automated recommendations into traceable, evidence-supported governance objects suitable for structured lifecycle assessment.